DISASTER RESPONSE *COORDINATION* SYSTEM

SQL Implementation and Query Demonstration

Data Warehousing and Data Management

Vishal H202549042 • Justin H202549071

BITS Pilani, Dubai

Table of Contents

Project Overview

This project converts the Disaster Response Coordination System ER diagram from Project 1 into a working relational database in MySQL. The schema captures the core operational entities involved in coordinating a disaster response: *disasters, affected regions, agencies, response teams, volunteers, shelters, and supplies.* Associative tables *(DisasterRegion, TeamDeployment, VolunteerAssignment, SupplyAllocation)* resolve the many-to-many relationships between these entities and carry the temporal attributes that describe when each event happened.

The implementation demonstrates the full range of SQL functionality required by the rubric: basic CRUD operations (CREATE, INSERT, UPDATE, DELETE, DROP, SELECT), filtering and aggregation (WHERE, LIKE, ORDER BY, GROUP BY, HAVING, UNION), three different join types, a correlated filter using a subquery with IN, and a closing showcase built around Common Table Expressions paired with window functions (RANK, DENSE_RANK, NTILE, PARTITION BY). Window functions and CTEs are not covered in the syllabus and serve as the project's *novel feature.*

Why This Database Matters

The Coordination Problem

When disaster strikes, the failure point is rarely the response itself. It is coordination between the people responding. Three patterns recur in every post-disaster review of the last decade:

Failure Pattern	What Goes Wrong	Operational Cost
Capacity blindness	A shelter fills up while a half-empty one operates two streets away	Evacuees turned away, secondary displacement
Skill mismatch	A surgeon hands out water bottles for three days because no one queries her skillset	Critical capability wasted, lives lost
Misrouted supplies	Trucks of blankets sent to a region that needed generators	Aid arrives late and wrong, donor confidence eroded
Duplicate effort	Two agencies deploy teams to the same area while another zone gets none	Visible inequity, public trust damaged

The root cause is structural. In a crisis, “information is generated *faster* than humans can process it”. Phone calls get missed, spreadsheets fork into incompatible versions, and the people who need a piece of information are rarely the same people who have it.

What a Database Adds

A relational database does not solve coordination on its own. What it provides is a single source of truth that is queryable in seconds rather than hours.

The schema in this project models exactly the entities that appear on every disaster-response checklist:

- **The disaster itself**, with its severity and timeline
- **The regions** it touches, with population and risk profile
- **The agencies** responding, by type and contact
- **The teams** those agencies field, by specialisation
- **The volunteers** staffing those teams, by skillset and clearance
- **The shelters** housing the displaced, with live capacity
- **The supplies** flowing to where they are needed

How the Database Would Be Used

The Daily Operations Dashboard

In practice, this database would sit behind a dashboard used by an operations centre. A coordinator opening the dashboard at the start of a shift needs answers to a small set of recurring questions;

Each one mapped to a specific query category:

Question	Query Type	Example from This Project
Which disasters are active right now?	*WHERE filter*	Disaster table filtered by status
Which shelters are dangerously full?	*NTILE bucketing*	Shelter stress tiers
Is any agency stretched too thin?	*GROUP BY count*	Teams per agency
Who is deployed where, and until when?	*INNER JOIN*	Full deployment roster
Which regions are still untouched?	*LEFT JOIN*	Region frequency, including zeros
Where are supplies sitting unused?	*RIGHT JOIN*	Supply utilization with zero rows kept
Who has handled severe disasters before?	*Subquery with IN*	Volunteers on severe-disaster teams

Window Functions Earn Their Place. The window function queries are the difference between a database that stores data and a database that helps coordinators decide things. Each one maps to a real operational question:

RANK and DENSE_RANK on supplies received per region. Not just a sorted list. The relative position is what drives the conversation about whether the distribution is fair, and the difference between RANK and DENSE_RANK matters when two regions tie.

NTILE bucketing on shelter occupancy. The tier groupings map directly to action categories:

Tier	Meaning	Action
1	Most stressed shelters (highest occupancy)	*Intervene now:* add capacity or relocate
2	Mid-tier shelters	*Monitor closely;* intervene if trend worsens
3	Slack capacity	Candidate for taking on *overflow*

PARTITION BY on per-agency volunteer leaderboards. Not one global ranking. Each agency lead gets the ranking that matters for their own roster, which is the level of granularity actually used in agency review meetings.

These mirror the kinds of tables that appear in real humanitarian-sector reporting: top-three most affected regions per disaster, top-five busiest volunteers per agency, capacity utilisation tier of every active shelter. Producing them with one query rather than a chain of exports and Excel pivots is what makes the database operationally useful, not just operationally interesting.

Limits of the Current Schema

It is worth being honest about what this implementation *does not do*.

The boundaries help the reader understand what would still need to be built before deploying for real:

Out of Scope	Why It Was Left Out	What Would Be Needed
Real-time geospatial data	Tracking storm movement is its own discipline	A separate GIS layer feeding location updates
Financial flows	Donor commitments and insurance claims are a domain on their own	A finance module with double-entry constraints
Audit logging	Out of scope for a course project	Row-level change tracking and immutable history
Access control	Single-user demo only	Role-based permissions, enforced at the database
Production scale	Sample data is intentionally small	Indexing strategy, query plan review at million-row scale

What the Project Does Demonstrate

What this project does demonstrate is sound at the level it covers:

- **The data model is correct.** Every entity in the original ERD is represented, every relationship is enforced through foreign keys, and the cardinality is preserved.
- **Integrity is enforced at the database, not the application.** CHECK constraints catch the obvious errors (occupancy exceeding capacity, deployment ending before it starts) so that no application bug can put the database into an invalid state.
- **The query patterns scale.** Each query in this document continues to work correctly as new disasters, regions, or teams are added. No schema change is required to add a sixth disaster or an eighth region.
- **The novel feature has a real purpose.** CTEs and window functions are not used here as a syntactic flourish. Each one answers a question that simpler SQL cannot answer cleanly.

Everything else is a layer that can be added on top of this foundation without disturbing what is here.

Schema Reference

A quick-reference summary of every table in the database, its primary key, foreign keys, and purpose. Use this as a map when reading the query sections that follow.

Core Entities

Table	Primary Key	Purpose
Disaster	disaster_id	One row per disaster event, with type, severity, and timeline
Region	region_id	Geographic regions that can be affected by disasters
Agency	agency_id	Organizations responding to disasters
Volunteer	volunteer_id	Individual responders, with skillset and clearance status
Supply	supply_id	Catalog of supply types (water, food, medical, shelter, equipment)

Dependent Entities

Table	Primary Key	Foreign Keys	Purpose
ResponseTeam	team_id	agency_id → Agency	Specialized teams fielded by agencies
Shelter	shelter_id	region_id → Region	Shelters with capacity and live occupancy

Junction Tables

These resolve the many-to-many relationships and carry the temporal attributes that describe when each event happened.

Table	Composite Primary Key	Foreign Keys	What It Links
DisasterRegion	(disaster_id, region_id)	Disaster, Region	Which regions are affected by which disasters, with impact level
TeamDeployment	(team_id, disaster_id, region_id)	ResponseTeam, DisasterRegion	When and where each team is deployed
VolunteerAssignment	(volunteer_id, team_id, assignment_start)	Volunteer, ResponseTeam	When each volunteer joined which team

SupplyAllocation	(supply_id, disaster_id, region_id, allocation_date)	Supply, DisasterRegion	When supplies were allocated to a disaster-region pair, and how much

Constraints in Force

The schema enforces business rules at the database layer rather than relying on the application:

Constraint	**Where**	**What It Prevents**
current_occupancy ≤ capacity	Shelter	A shelter being recorded as over capacity
deployment_end > deployment_start	TeamDeployment	A deployment with a non-positive duration
background_check_status NOT NULL	Volunteer	A volunteer being entered without a clearance record
Foreign keys throughout	All junction tables	Orphan rows referencing non-existent disasters, regions, teams, or supplies

Cardinality at a Glance

The flow of relationships, top to bottom:

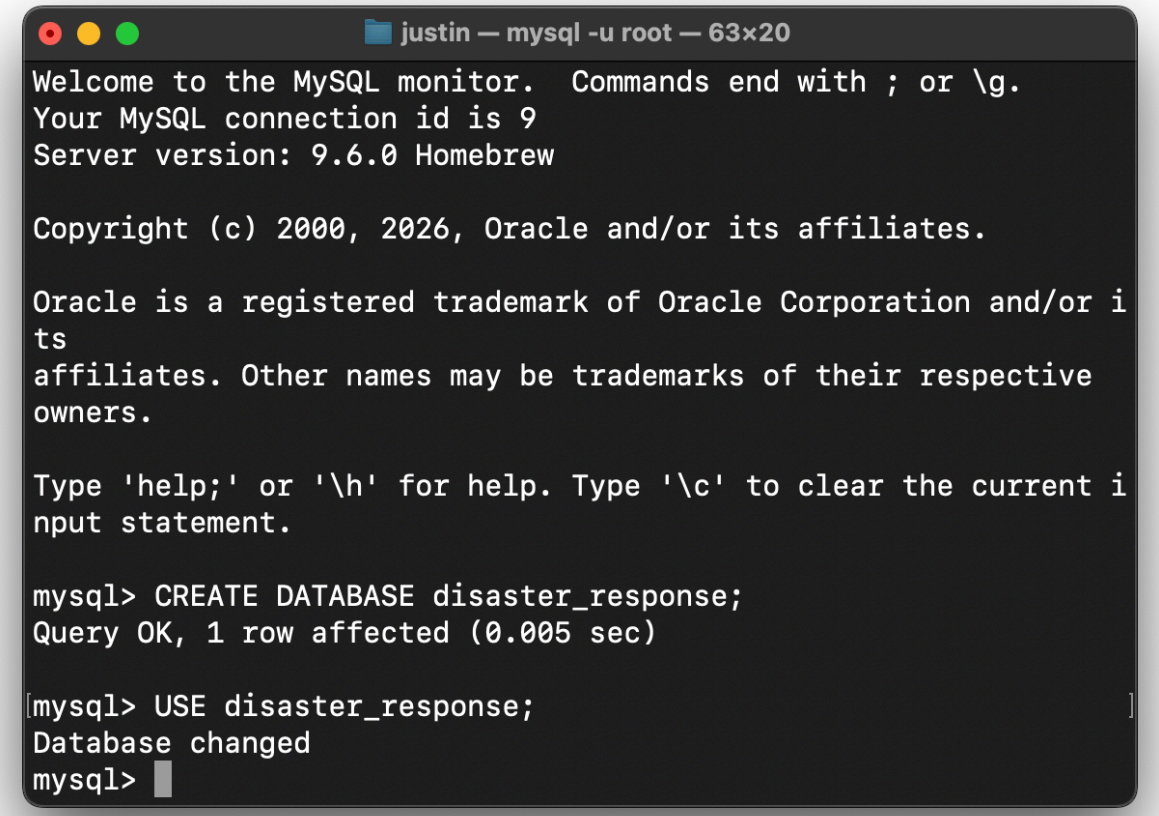
- One **Agency** has many **ResponseTeams**
- One **Region** has many **Shelters**
- One **Disaster** affects many **Regions** (and vice versa) through **DisasterRegion**
- One **ResponseTeam** is deployed to many disaster-region pairs through **TeamDeployment**
- One **Volunteer** can join many **ResponseTeams** over time through **VolunteerAssignment**
- One **Supply** type can be allocated to many disaster-region pairs through **SupplyAllocation**

Database and Schema Setup

Creating the Database

The database is created with a single statement and selected for use so that subsequent table definitions land in the correct schema.

```
CREATE DATABASE disaster_response;
USE disaster_response;
```



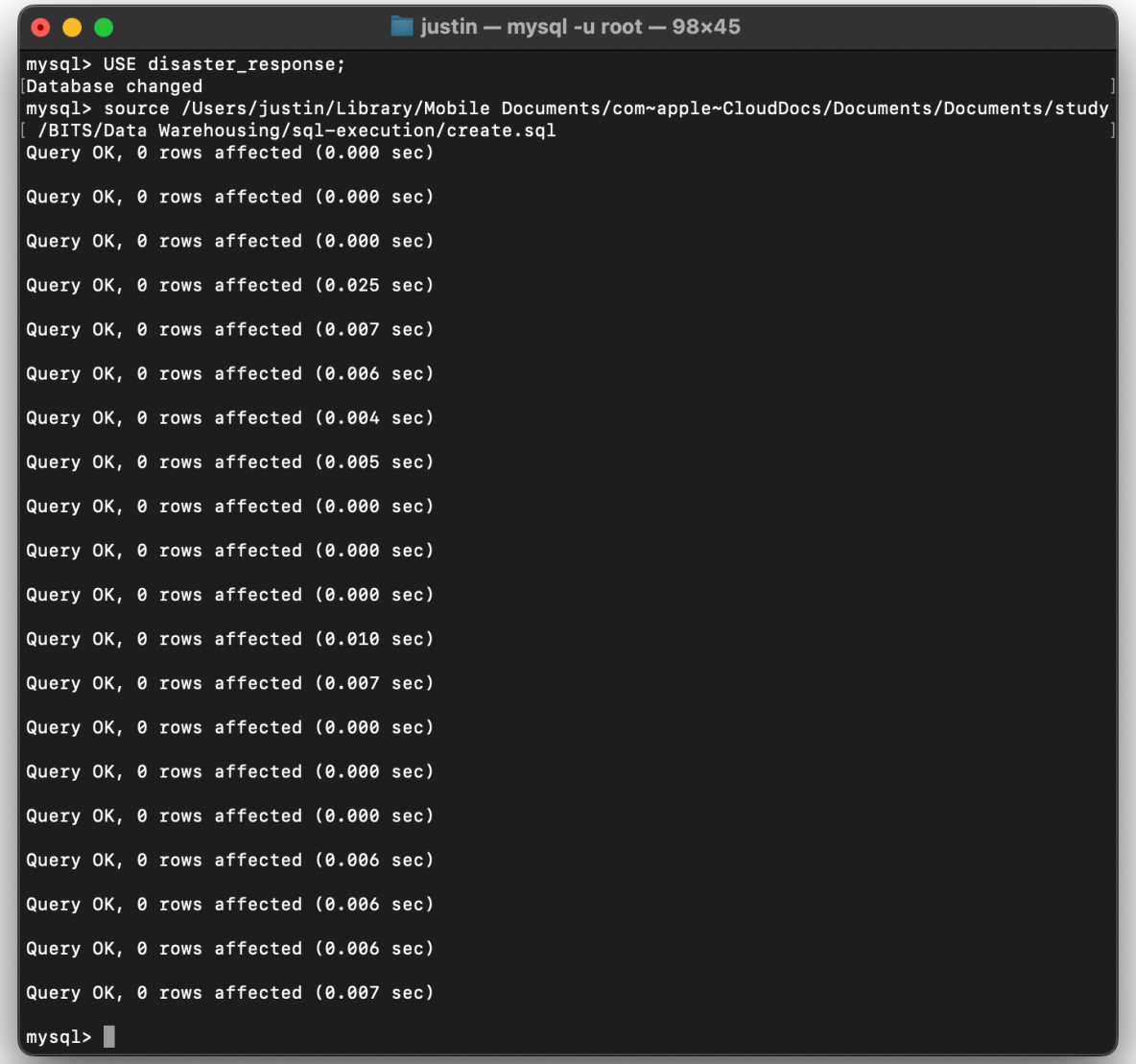
Output · schema panel after running the script

Creating the Tables

Tables are created in dependency order. Core entities with no foreign keys come first, followed by tables that reference them (ResponseTeam references Agency, Shelter references Region), and finally the four junction tables that resolve the many-to-many relationships. The CHECK constraint on Shelter ensures that occupancy can never exceed capacity, and the CHECK on TeamDeployment ensures that a deployment end time is always after its start time. Background check status on Volunteer is marked NOT NULL because the original ERD treated it as a mandatory data integrity rule.

```
-- Core entities with no foreign key dependencies

CREATE TABLE Disaster (
    disaster_id     INT PRIMARY KEY,
    name            VARCHAR(100) NOT NULL,
    disaster_type   VARCHAR(50),
    severity_level  VARCHAR(20),
    start_date      DATE,
    end_date        DATE,
    status          VARCHAR(20)
);

CREATE TABLE Region (
    region_id   INT PRIMARY KEY,
    name        VARCHAR(100) NOT NULL,
    population  INT,
    risk_level  VARCHAR(20)
);

CREATE TABLE Agency (
    agency_id    INT PRIMARY KEY,
    name         VARCHAR(100) NOT NULL,
    agency_type  VARCHAR(50),
    contact_info VARCHAR(150)
);

CREATE TABLE Volunteer (
    volunteer_id            INT PRIMARY KEY,
    name                    VARCHAR(100) NOT NULL,
    skillset                VARCHAR(150),
    availability_status     VARCHAR(30),
    background_check_status VARCHAR(20) NOT NULL
);

CREATE TABLE Supply (
    supply_id  INT PRIMARY KEY,
    name       VARCHAR(100) NOT NULL,
    category   VARCHAR(50),
    unit_type  VARCHAR(20)
);

-- Entities with one foreign key

CREATE TABLE ResponseTeam (
    team_id           INT PRIMARY KEY,
    agency_id         INT,
    specialization    VARCHAR(100),
    deployment_status VARCHAR(30),
    FOREIGN KEY (agency_id) REFERENCES Agency(agency_id)
);
```

```sql
CREATE TABLE Shelter (
    shelter_id        INT PRIMARY KEY,
    region_id         INT,
    capacity          INT,
    current_occupancy INT,
    status            VARCHAR(30),
    FOREIGN KEY (region_id) REFERENCES Region(region_id),
    CHECK (current_occupancy <= capacity)
);

-- Junction tables resolving many-to-many relationships

CREATE TABLE DisasterRegion (
    disaster_id      INT,
    region_id        INT,
    impact_level     VARCHAR(20),
    evacuation_order VARCHAR(20),
    PRIMARY KEY (disaster_id, region_id),
    FOREIGN KEY (disaster_id) REFERENCES Disaster(disaster_id),
    FOREIGN KEY (region_id)   REFERENCES Region(region_id)
);

CREATE TABLE TeamDeployment (
    team_id          INT,
    disaster_id      INT,
    region_id        INT,
    deployment_start DATETIME,
    deployment_end   DATETIME,
    PRIMARY KEY (team_id, disaster_id, region_id),
    FOREIGN KEY (team_id) REFERENCES ResponseTeam(team_id),
    FOREIGN KEY (disaster_id, region_id)
        REFERENCES DisasterRegion(disaster_id, region_id),
    CHECK (deployment_end > deployment_start)
);

CREATE TABLE VolunteerAssignment (
    volunteer_id     INT,
    team_id          INT,
    assignment_start DATETIME,
    assignment_end   DATETIME,
    PRIMARY KEY (volunteer_id, team_id, assignment_start),
    FOREIGN KEY (volunteer_id) REFERENCES Volunteer(volunteer_id),
    FOREIGN KEY (team_id)      REFERENCES ResponseTeam(team_id)
);

CREATE TABLE SupplyAllocation (
    supply_id          INT,
    disaster_id        INT,
    region_id          INT,
    allocation_date    DATE,
    quantity_allocated INT,
    PRIMARY KEY (supply_id, disaster_id, region_id, allocation_date),
    FOREIGN KEY (supply_id) REFERENCES Supply(supply_id),
    FOREIGN KEY (disaster_id, region_id)
        REFERENCES DisasterRegion(disaster_id, region_id)
);
```



Output · table creation confirmation

Sample Data Insertion

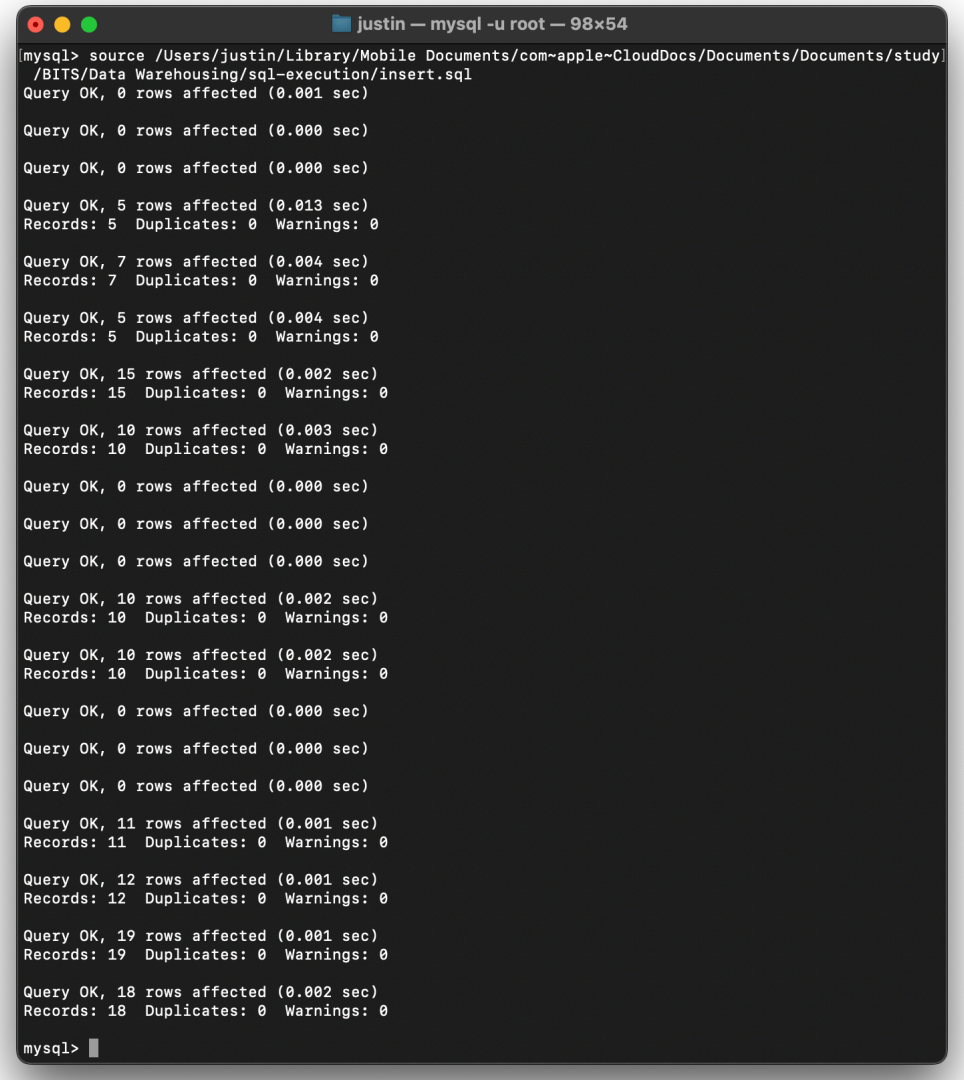
Sample data covers five disasters affecting seven regions of the Philippines, five coordinating agencies, ten response teams, fifteen volunteers, ten supply types, and ten shelters. The volume is deliberately chosen to produce a clean three-way split for the NTILE bucketing query later in the project. All foreign key relationships are satisfied; insertions follow the same parent-before-child order as the table creation.

Core Entity Inserts

```
INSERT INTO Disaster VALUES
(1, 'Typhoon Yagi',          'Typhoon',    'High',     '2024-09-02', '2024-09-08', 'Closed'),
(2, 'Luzon Earthquake',      'Earthquake', 'Severe',   '2024-11-15', '2024-11-22', 'Closed'),
(3, 'Mindanao Floods',       'Flood',      'Moderate', '2025-01-10', '2025-01-18', 'Closed'),
(4, 'Mt. Kanlaon Eruption',  'Volcanic',   'High',     '2025-06-03', NULL,         'Active'),
(5, 'Visayas Wildfire',      'Wildfire',   'Moderate', '2025-04-12', '2025-04-20', 'Closed');

INSERT INTO Region VALUES
(1, 'Metro Manila',     13500000, 'High'),
(2, 'Cebu',              3200000, 'Medium'),
(3, 'Davao',             1800000, 'Medium'),
(4, 'Cagayan Valley',    1200000, 'High'),
(5, 'Negros Occidental', 2600000, 'Medium'),
(6, 'Iloilo',            2050000, 'Low'),
(7, 'Bicol',             1900000, 'High');

INSERT INTO Agency VALUES
(1, 'Philippine Red Cross',   'NGO',          'redcross@ph.org / 02-8527-0000'),
(2, 'NDRRMC',                 'Government',   'ops@ndrrmc.gov.ph / 02-8911-1406'),
(3, 'World Vision PH',        'NGO',          'info@worldvision.ph'),
(4, 'AFP Engineering Corps',  'Military',     'afp.eng@mil.ph'),
(5, 'UN OCHA Philippines',    'International','ocha.ph@un.org');

INSERT INTO Volunteer VALUES
(1,  'Maria Santos',     'Medical, First Aid',          'Available',   'Cleared'),
(2,  'Juan Dela Cruz',   'Logistics, Driving',          'Deployed',    'Cleared'),
(3,  'Ana Reyes',        'Medical, Triage',             'Deployed',    'Cleared'),
(4,  'Carlos Lim',       'Search and Rescue',           'Available',   'Cleared'),
(5,  'Liza Mendoza',     'Counseling, Child Care',      'Deployed',    'Cleared'),
(6,  'Ramon Tan',        'Engineering, Heavy Equipment','Deployed',    'Cleared'),
(7,  'Grace Villanueva', 'Translation, Coordination',   'Available',   'Cleared'),
(8,  'Dennis Cruz',      'Search and Rescue, Diving',   'Deployed',    'Cleared'),
(9,  'Patricia Ong',     'Medical, Surgery',            'Unavailable', 'Cleared'),
(10, 'Mark Aquino',      'Logistics',                   'Deployed',    'Pending'),
(11, 'Sofia Garcia',     'Counseling',                  'Available',   'Cleared'),
(12, 'Rafael Bautista',  'Engineering',                 'Deployed',    'Cleared'),
(13, 'Isabel Domingo',   'Medical, Pediatrics',         'Deployed',    'Cleared'),
(14, 'Noel Ramos',       'Drone Operations',            'Available',   'Cleared'),
(15, 'Beatriz Cruz',     'Logistics, Inventory',        'Deployed',    'Cleared');

INSERT INTO Supply VALUES
(1, 'Bottled Water',      'Water',     'Liters'),
(2, 'Rice (25kg sacks)',  'Food',      'Sacks'),
(3, 'Canned Goods',       'Food',      'Boxes'),
(4, 'First Aid Kits',     'Medical',   'Kits'),
(5, 'Antibiotics',        'Medical',   'Boxes'),
(6, 'Blankets',           'Shelter',   'Pieces'),
(7, 'Tarpaulin Sheets',   'Shelter',   'Pieces'),
```

```
(8, 'Generators',         'Equipment', 'Units'),
(9, 'Flashlights',        'Equipment', 'Units'),
(10,'Hygiene Kits',       'Medical',   'Kits');

-- Dependent Entities and Junction Tables

INSERT INTO ResponseTeam VALUES
(1,  1, 'Medical Triage',         'Deployed'),
(2,  1, 'Search and Rescue',      'Deployed'),
(3,  2, 'Logistics and Supply',   'Deployed'),
(4,  2, 'Damage Assessment',      'Standby'),
(5,  3, 'Child Protection',       'Deployed'),
(6,  3, 'Food Distribution',      'Deployed'),
(7,  4, 'Engineering and Debris', 'Deployed'),
(8,  4, 'Heavy Equipment Ops',    'Standby'),
(9,  5, 'Coordination Cell',      'Deployed'),
(10, 1, 'Mental Health Support',  'Standby');

INSERT INTO Shelter VALUES
(1,  1, 500, 480, 'Operational'),
(2,  1, 300, 295, 'Operational'),
(3,  2, 400, 250, 'Operational'),
(4,  3, 200, 180, 'Operational'),
(5,  4, 350, 350, 'Full'),
(6,  5, 600, 410, 'Operational'),
(7,  6, 250,  90, 'Operational'),
(8,  7, 450, 445, 'Operational'),
(9,  2, 300, 120, 'Operational'),
(10, 7, 200,   0, 'Closed');

INSERT INTO DisasterRegion VALUES
(1, 1, 'Severe',   'Mandatory'),  (1, 4, 'High',     'Mandatory'),
(1, 7, 'High',     'Voluntary'),  (2, 1, 'High',     'Voluntary'),
(2, 4, 'Severe',   'Mandatory'),  (3, 3, 'Moderate', 'Voluntary'),
(3, 5, 'High',     'Mandatory'),  (4, 5, 'Severe',   'Mandatory'),
(4, 6, 'Moderate', 'Voluntary'),  (5, 2, 'Moderate', 'Voluntary'),
(5, 6, 'Low',      'None');

INSERT INTO TeamDeployment VALUES
(1,  1, 1, '2024-09-02 06:00:00', '2024-09-09 18:00:00'),
(2,  1, 4, '2024-09-02 08:00:00', '2024-09-10 12:00:00'),
(3,  1, 7, '2024-09-03 07:00:00', '2024-09-09 20:00:00'),
(1,  2, 1, '2024-11-15 10:00:00', '2024-11-25 18:00:00'),
(4,  2, 4, '2024-11-15 14:00:00', '2024-11-23 12:00:00'),
(7,  2, 4, '2024-11-16 06:00:00', '2024-11-30 18:00:00'),
(5,  3, 5, '2025-01-10 09:00:00', '2025-01-20 17:00:00'),
(6,  3, 3, '2025-01-11 06:00:00', '2025-01-19 18:00:00'),
(7,  4, 5, '2025-06-03 12:00:00', '2025-07-15 18:00:00'),
(8,  4, 6, '2025-06-04 08:00:00', '2025-07-10 18:00:00'),
(9,  5, 2, '2025-04-12 09:00:00', '2025-04-21 18:00:00'),
(2,  5, 6, '2025-04-13 07:00:00', '2025-04-20 18:00:00');

INSERT INTO VolunteerAssignment VALUES
(1,  1, '2024-09-02 06:00:00', '2024-09-09 18:00:00'),
(3,  1, '2024-09-02 06:00:00', '2024-09-09 18:00:00'),
(4,  2, '2024-09-02 08:00:00', '2024-09-10 12:00:00'),
(8,  2, '2024-09-02 08:00:00', '2024-09-10 12:00:00'),
(2,  3, '2024-09-03 07:00:00', '2024-09-09 20:00:00'),
(10, 3, '2024-09-03 07:00:00', '2024-09-09 20:00:00'),
(15, 3, '2024-09-03 07:00:00', '2024-09-09 20:00:00'),
(1,  1, '2024-11-15 10:00:00', '2024-11-25 18:00:00'),
(13, 1, '2024-11-15 10:00:00', '2024-11-25 18:00:00'),
(6,  7, '2024-11-16 06:00:00', '2024-11-30 18:00:00'),
(12, 7, '2024-11-16 06:00:00', '2024-11-30 18:00:00'),
(5,  5, '2025-01-10 09:00:00', '2025-01-20 17:00:00'),
```

```sql
(11, 5, '2025-01-10 09:00:00', '2025-01-20 17:00:00'),
(2,  6, '2025-01-11 06:00:00', '2025-01-19 18:00:00'),
(6,  7, '2025-06-03 12:00:00', '2025-07-15 18:00:00'),
(12, 7, '2025-06-03 12:00:00', '2025-07-15 18:00:00'),
(14, 8, '2025-06-04 08:00:00', '2025-07-10 18:00:00'),
(7,  9, '2025-04-12 09:00:00', '2025-04-21 18:00:00'),
(3,  2, '2025-04-13 07:00:00', '2025-04-20 18:00:00');

INSERT INTO SupplyAllocation VALUES
(1,  1, 1, '2024-09-03', 50000),  (2,  1, 1, '2024-09-03',  2000),
(4,  1, 1, '2024-09-03',   500),  (1,  1, 4, '2024-09-04', 30000),
(6,  1, 4, '2024-09-04',  1500),  (7,  1, 7, '2024-09-04',   800),
(1,  2, 4, '2024-11-16', 60000),  (5,  2, 4, '2024-11-16',   200),
(8,  2, 4, '2024-11-17',    25),  (2,  2, 1, '2024-11-17',  3000),
(1,  3, 5, '2025-01-11', 40000),  (3,  3, 5, '2025-01-11',  1200),
(10, 3, 3, '2025-01-12',   400),  (7,  4, 5, '2025-06-04',  1800),
(9,  4, 5, '2025-06-04',   600),  (6,  4, 6, '2025-06-05',  1000),
(4,  5, 2, '2025-04-13',   250),  (3,  5, 6, '2025-04-14',   500);
```

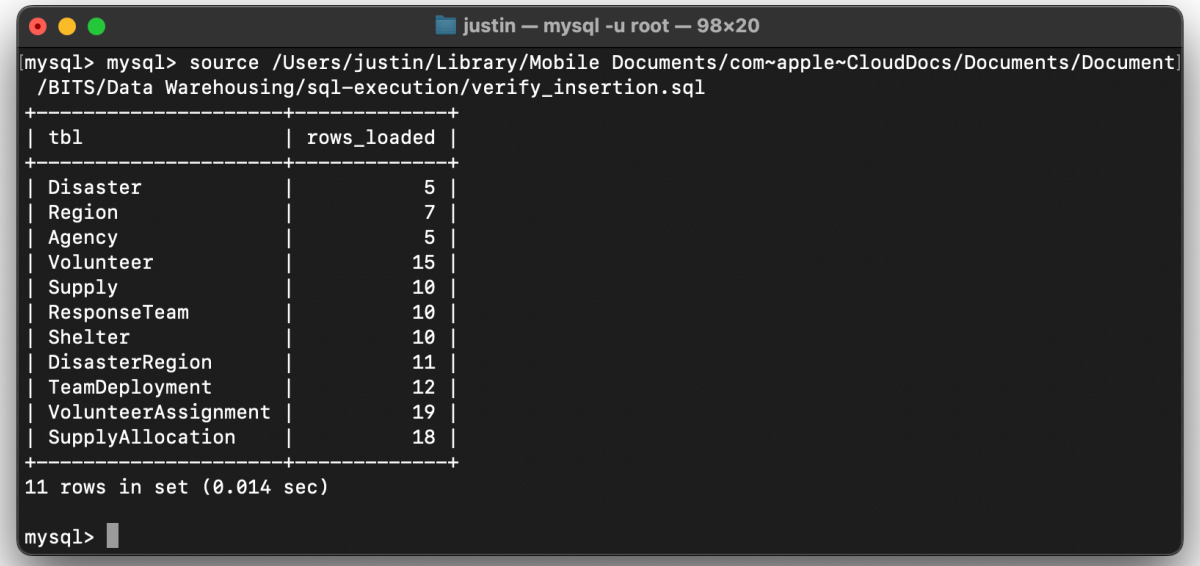


Output · inserts complete with no foreign key errors

Verifying the Load with UNION

A single multi-part query stitched together with UNION ALL returns the row count for every table in one result set. UNION ALL is preferred over UNION here because the rows are guaranteed unique (different table names) and UNION ALL avoids the unnecessary deduplication pass. This step doubles as the project's UNION requirement.

```sql
SELECT 'Disaster' AS tbl, COUNT(*) AS rows_loaded FROM Disaster UNION ALL
SELECT 'Region',          COUNT(*) FROM Region UNION ALL
SELECT 'Agency',          COUNT(*) FROM Agency UNION ALL
SELECT 'Volunteer',       COUNT(*) FROM Volunteer UNION ALL
SELECT 'Supply',          COUNT(*) FROM Supply UNION ALL
SELECT 'ResponseTeam',    COUNT(*) FROM ResponseTeam UNION ALL
SELECT 'Shelter',         COUNT(*) FROM Shelter UNION ALL
SELECT 'DisasterRegion',  COUNT(*) FROM DisasterRegion UNION ALL
SELECT 'TeamDeployment',  COUNT(*) FROM TeamDeployment UNION ALL
SELECT 'VolunteerAssignment', COUNT(*) FROM VolunteerAssignment UNION ALL
SELECT 'SupplyAllocation',    COUNT(*) FROM SupplyAllocation;
```



Output · 11 rows confirming load: 5, 7, 5, 15, 10, 10, 10, 11, 12, 19, 18

Filtering and Sorting

WHERE clause active disasters

WHERE narrows the result set to rows that satisfy a predicate. It isolates the disasters whose status is currently Active, which would be the operational view a coordinator needs at the start of each shift.

```
SELECT disaster_id, name, disaster_type, severity_level, status
FROM Disaster
WHERE status = 'Active';
```

LIKE wildcard medical-skilled volunteers

LIKE combined with the percent wildcard finds any volunteer whose skillset string contains the substring Medical, regardless of what surrounds it. This catches Medical, First Aid as well as Medical, Triage and Medical, Surgery without the query needing to know the exact phrasing of every record.

```
SELECT volunteer_id, name, skillset, availability_status
FROM Volunteer
WHERE skillset LIKE '%Medical%';
```

ORDER BY descending: regions by population

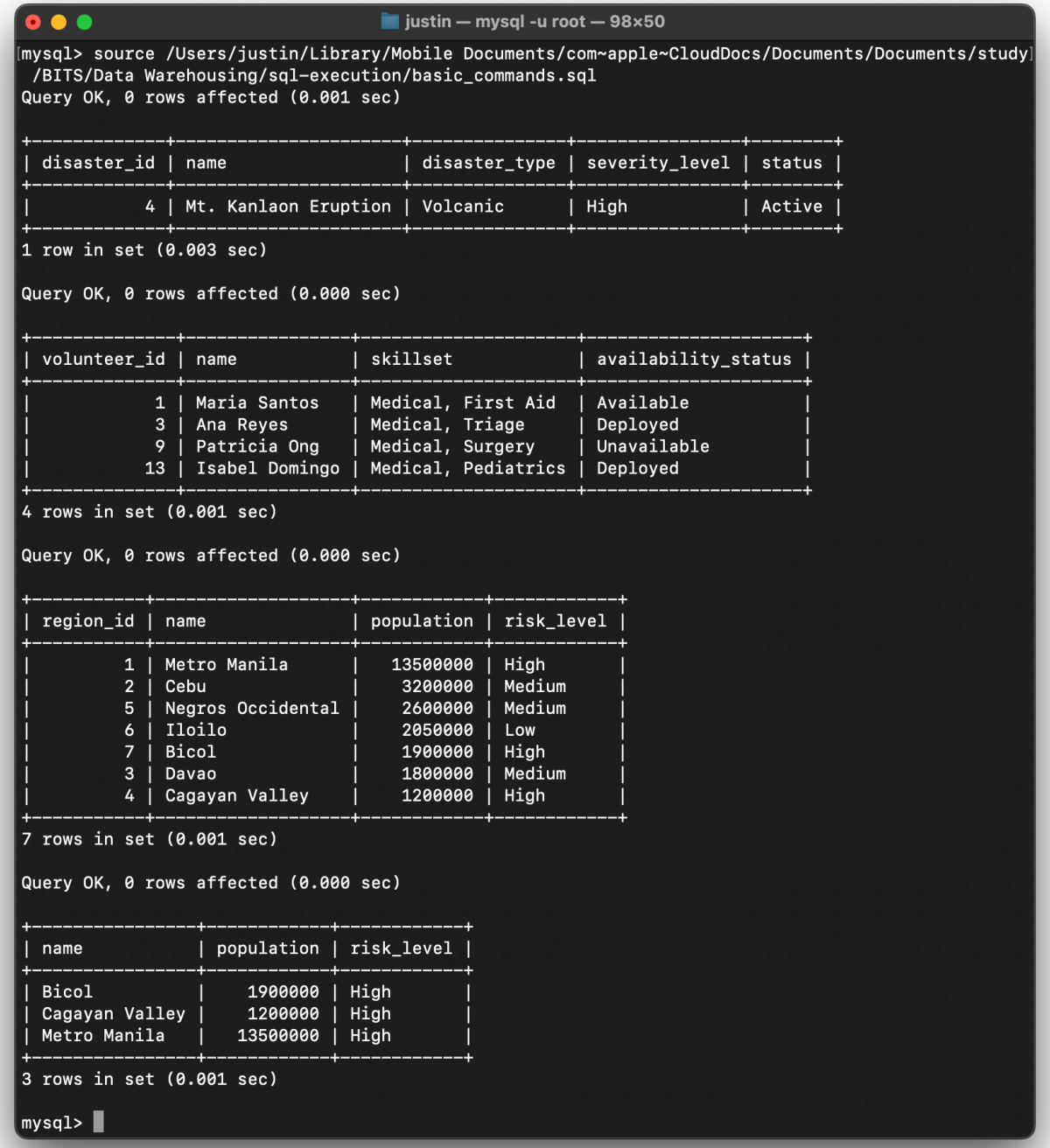
ORDER BY with the DESC keyword returns regions from largest to smallest population. This is the kind of ordering used to prioritize resource allocation when human lives at risk are roughly proportional to people exposed.

```
SELECT region_id, name, population, risk_level
FROM Region
ORDER BY population DESC;
```

ORDER BY ascending with WHERE

Combining WHERE and ORDER BY ASC produces an alphabetised list of just the high-risk regions. ASC is the default sort direction in SQL, but stating it explicitly makes the intent unambiguous.

```
SELECT name, population, risk_level
FROM Region
WHERE risk_level = 'High'
ORDER BY name ASC;
```



Output · combined results for the four queries above

Aggregation

GROUP BY response teams per agency

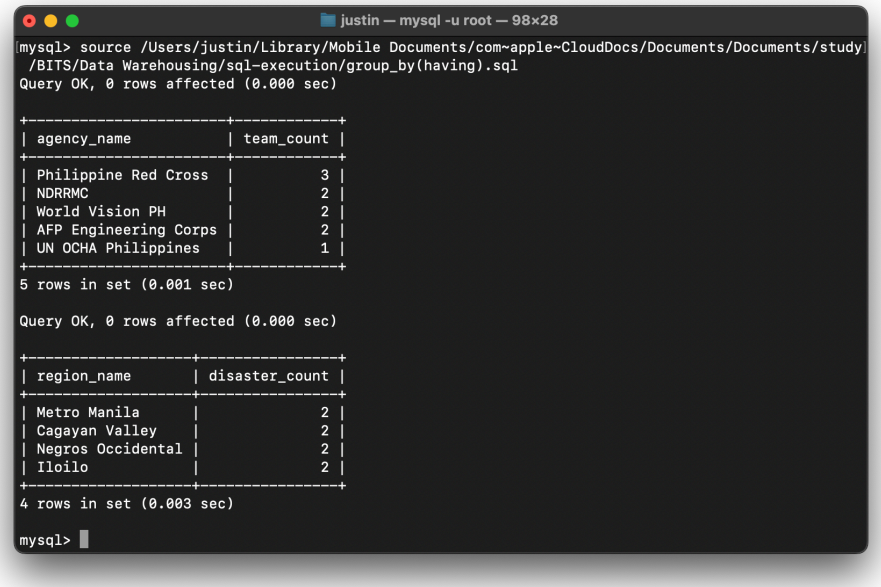
GROUP BY collapses rows that share an agency_id into a single output row, with COUNT calculating how many teams each agency operates. The LEFT JOIN ensures that agencies with zero teams still appear in the result with a count of 0; an INNER JOIN would silently exclude them and hide a potentially important operational gap.

```sql
SELECT a.name AS agency_name, COUNT(rt.team_id) AS team_count
FROM Agency a
LEFT JOIN ResponseTeam rt ON a.agency_id = rt.agency_id
GROUP BY a.agency_id, a.name;
```

HAVING regions hit by multiple disasters

HAVING filters rows after aggregation, which is necessary because WHERE runs before grouping and cannot reference COUNT or other aggregate functions. This query returns only the regions that have been affected by more than one disaster, sorted by frequency.

```sql
SELECT r.name AS region_name, COUNT(dr.disaster_id) AS disaster_count
FROM Region r
JOIN DisasterRegion dr ON r.region_id = dr.region_id
GROUP BY r.region_id, r.name
HAVING COUNT(dr.disaster_id) > 1
ORDER BY disaster_count DESC;
```



Output · combined results for GROUP BY and HAVING

Joining Tables

INNER JOIN deployment roster across disasters

An INNER JOIN returns only rows where every table in the join chain has a matching record. Here the result is a complete operational roster showing which agency's team was deployed to which disaster in which region, along with the dates of deployment. Any deployment missing a team, agency, disaster, or region link would be excluded, which is the correct behaviour for an audit-grade roster.

```
SELECT
    d.name           AS disaster_name,
    r.name           AS region_name,
    a.name           AS agency_name,
    rt.specialization,
    td.deployment_start,
    td.deployment_end
FROM TeamDeployment td
INNER JOIN ResponseTeam rt ON td.team_id     = rt.team_id
INNER JOIN Agency       a  ON rt.agency_id   = a.agency_id
INNER JOIN Disaster     d  ON td.disaster_id = d.disaster_id
INNER JOIN Region       r  ON td.region_id   = r.region_id
ORDER BY td.deployment_start;
```

LEFT JOIN region disaster frequency

A LEFT JOIN keeps every row from the left table (Region) regardless of whether a matching row exists in the right table (DisasterRegion). This produces a complete frequency report including regions that have been spared so far, which would have been lost with an INNER JOIN. Such regions show a count of 0.

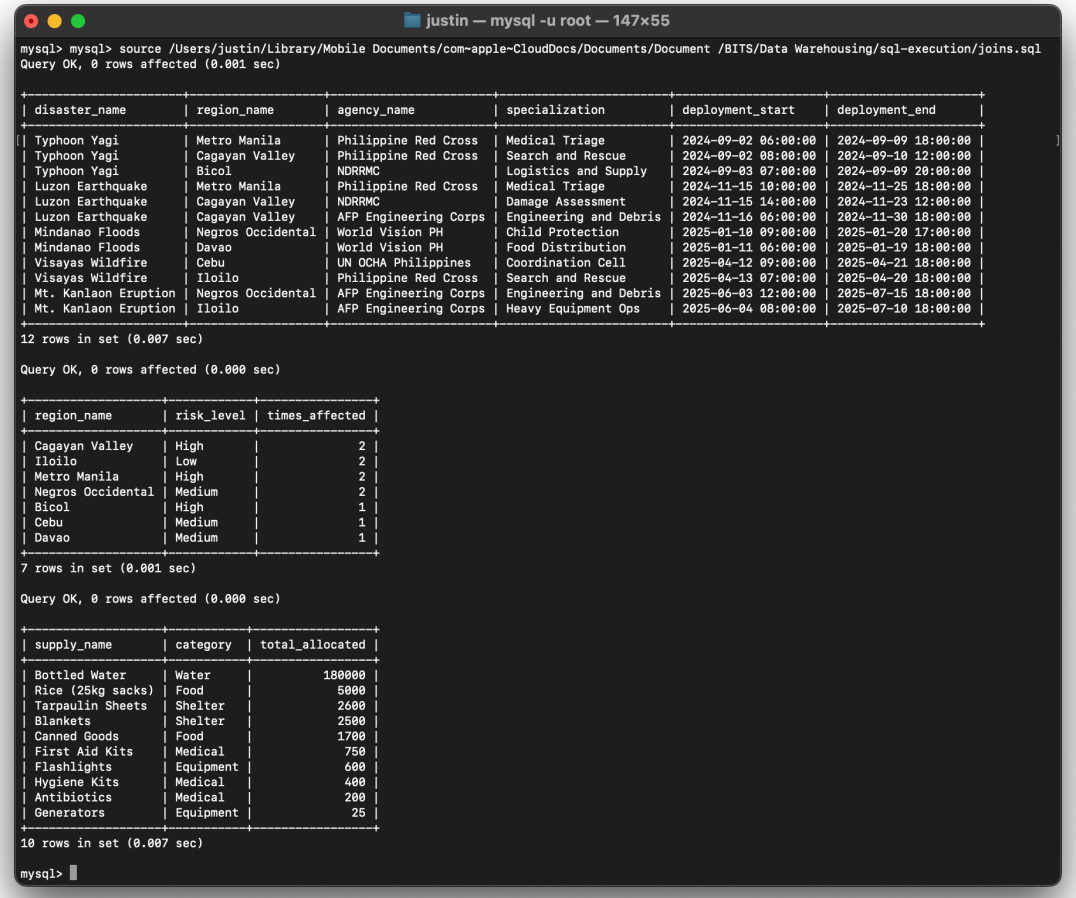
```
SELECT
    r.name              AS region_name,
    r.risk_level,
    COUNT(dr.disaster_id) AS times_affected
FROM Region r
LEFT JOIN DisasterRegion dr ON r.region_id = dr.region_id
GROUP BY r.region_id, r.name, r.risk_level
ORDER BY times_affected DESC, r.name;
```

RIGHT JOIN supply utilization with unused items

A RIGHT JOIN behaves as the mirror image of LEFT JOIN, retaining every row from the right table (Supply) even when no matching allocation exists. The result is a utilization report that surfaces unused supply types as well as well-utilized ones. COALESCE converts the NULL totals from unused supplies into a clean 0.

```sql
SELECT
    s.name                          AS supply_name,
    s.category,
    COALESCE(SUM(sa.quantity_allocated), 0) AS total_allocated
FROM SupplyAllocation sa
RIGHT JOIN Supply s ON sa.supply_id = s.supply_id
GROUP BY s.supply_id, s.name, s.category
ORDER BY total_allocated DESC;
```

Note: MySQL does not support FULL OUTER JOIN natively. The equivalent result can be produced by combining LEFT JOIN and RIGHT JOIN with UNION, but this project does not require it.



```
justin — mysql -u root — 147×55
mysql> mysql> source /Users/justin/Library/Mobile Documents/com~apple~CloudDocs/Documents/Document /BITS/Data Warehousing/sql-execution/joins.sql
Query OK, 0 rows affected (0.001 sec)

+---------------------+-------------------+---------------------+---------------------+---------------------+---------------------+
| disaster_name       | region_name       | agency_name         | specialization      | deployment_start    | deployment_end      |
+---------------------+-------------------+---------------------+---------------------+---------------------+---------------------+
| Typhoon Yagi        | Metro Manila      | Philippine Red Cross | Medical Triage      | 2024-09-02 06:00:00 | 2024-09-09 18:00:00 |
| Typhoon Yagi        | Cagayan Valley    | Philippine Red Cross | Search and Rescue   | 2024-09-02 08:00:00 | 2024-09-10 12:00:00 |
| Typhoon Yagi        | Bicol             | NDRRMC              | Logistics and Supply | 2024-09-03 07:00:00 | 2024-09-09 20:00:00 |
| Luzon Earthquake    | Metro Manila      | Philippine Red Cross | Medical Triage      | 2024-11-15 10:00:00 | 2024-11-25 18:00:00 |
| Luzon Earthquake    | Cagayan Valley    | NDRRMC              | Damage Assessment   | 2024-11-15 14:00:00 | 2024-11-23 12:00:00 |
| Luzon Earthquake    | Cagayan Valley    | AFP Engineering Corps | Engineering and Debris | 2024-11-16 06:00:00 | 2024-11-30 18:00:00 |
| Mindanao Floods     | Negros Occidental | World Vision PH     | Child Protection    | 2025-01-10 09:00:00 | 2025-01-20 17:00:00 |
| Mindanao Floods     | Davao             | World Vision PH     | Food Distribution   | 2025-01-11 06:00:00 | 2025-01-19 18:00:00 |
| Visayas Wildfire    | Cebu              | UN OCHA Philippines | Coordination Cell   | 2025-04-12 09:00:00 | 2025-04-21 18:00:00 |
| Visayas Wildfire    | Iloilo            | Philippine Red Cross | Search and Rescue   | 2025-04-13 07:00:00 | 2025-04-20 18:00:00 |
| Mt. Kanlaon Eruption | Negros Occidental | AFP Engineering Corps | Engineering and Debris | 2025-06-03 12:00:00 | 2025-07-15 18:00:00 |
| Mt. Kanlaon Eruption | Iloilo            | AFP Engineering Corps | Heavy Equipment Ops | 2025-06-04 08:00:00 | 2025-07-10 18:00:00 |
+---------------------+-------------------+---------------------+---------------------+---------------------+---------------------+
12 rows in set (0.007 sec)

Query OK, 0 rows affected (0.000 sec)

+-------------------+------------+----------------+
| region_name       | risk_level | times_affected |
+-------------------+------------+----------------+
| Cagayan Valley    | High       |              2 |
| Iloilo            | Low        |              2 |
| Metro Manila      | High       |              2 |
| Negros Occidental | Medium     |              2 |
| Bicol             | High       |              1 |
| Cebu              | Medium     |              1 |
| Davao             | Medium     |              1 |
+-------------------+------------+----------------+
7 rows in set (0.001 sec)

Query OK, 0 rows affected (0.000 sec)

+-------------------+-----------+-----------------+
| supply_name       | category  | total_allocated |
+-------------------+-----------+-----------------+
| Bottled Water     | Water     |          180000 |
| Rice (25kg sacks) | Food      |            5000 |
| Tarpaulin Sheets  | Shelter   |            2600 |
| Blankets          | Shelter   |            2500 |
| Canned Goods      | Food      |            1700 |
| First Aid Kits    | Medical   |             750 |
| Flashlights       | Equipment |             600 |
| Hygiene Kits      | Medical   |             400 |
| Antibiotics       | Medical   |             200 |
| Generators        | Equipment |              25 |
+-------------------+-----------+-----------------+
10 rows in set (0.007 sec)

mysql>
```

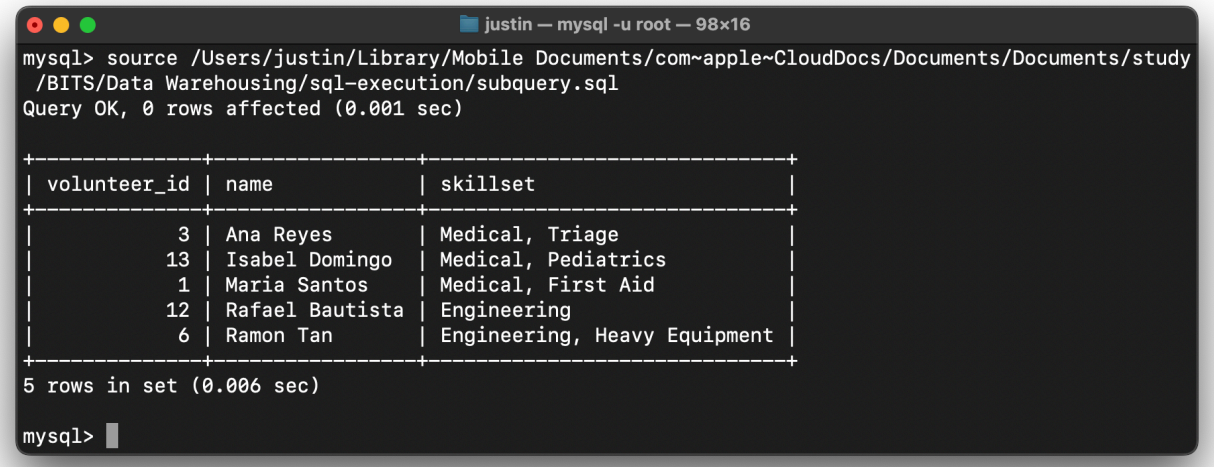
Output · combined results for the three join queries

Subquery

Volunteers who worked on Severe disasters

A subquery is a query nested inside another query. The inner SELECT returns a list of every team_id that was deployed to a disaster classified as Severe. The outer query then uses IN to find every volunteer whose assignments include any of those teams. DISTINCT removes duplicates that arise when a single volunteer worked multiple qualifying teams. This pattern is the standard approach when the filter condition itself requires its own query to compute.

```sql
SELECT DISTINCT v.volunteer_id, v.name, v.skillset
FROM Volunteer v
JOIN VolunteerAssignment va ON v.volunteer_id = va.volunteer_id
WHERE va.team_id IN (
    SELECT td.team_id
    FROM TeamDeployment td
    JOIN Disaster d ON td.disaster_id = d.disaster_id
    WHERE d.severity_level = 'Severe'
)
ORDER BY v.name;
```



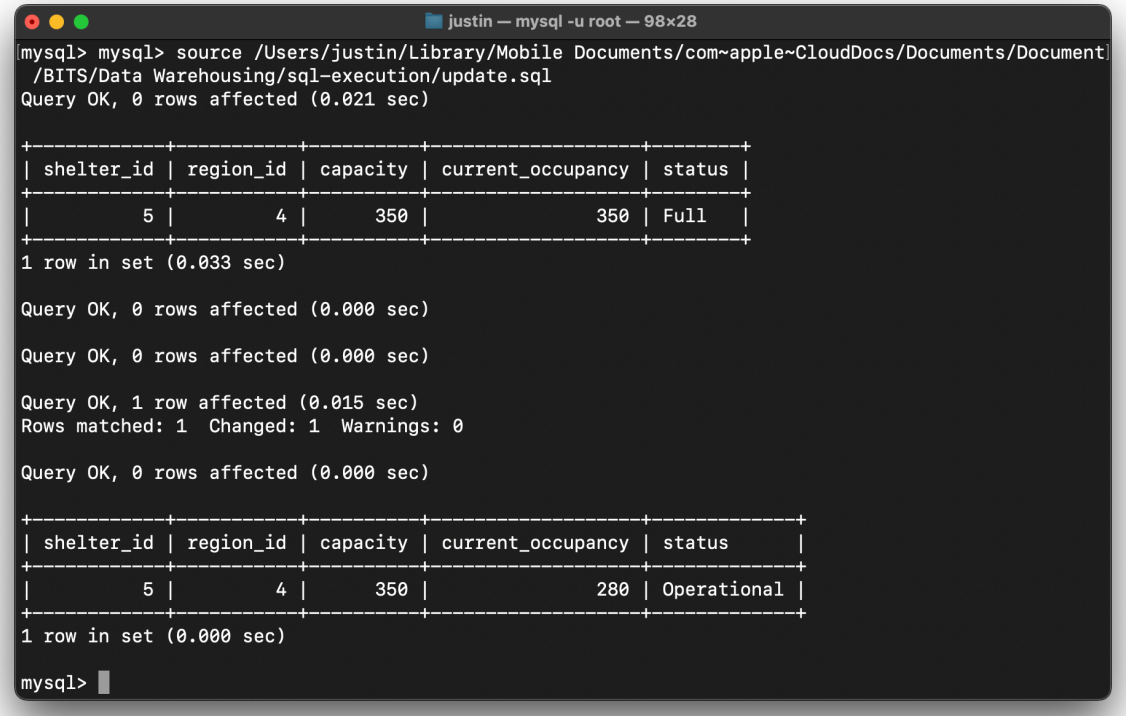
Output · volunteers from teams deployed to Severe disasters

Modifying Data

UPDATE Reducing shelter occupancy after relocation

Each modification follows a verify-modify-verify pattern: the first SELECT captures the row's current state, the UPDATE applies the change, and the second SELECT confirms the new state. Shelter 5 was previously at full capacity (350 of 350) with status Full; once evacuees were relocated, occupancy dropped to 280 and the status was reset to Operational.

```sql
-- Before
SELECT shelter_id, region_id, capacity, current_occupancy, status
FROM Shelter
WHERE shelter_id = 5;

-- Update
UPDATE Shelter
SET current_occupancy = 280,
    status = 'Operational'
WHERE shelter_id = 5;

-- After
SELECT shelter_id, region_id, capacity, current_occupancy, status
FROM Shelter
WHERE shelter_id = 5;
```

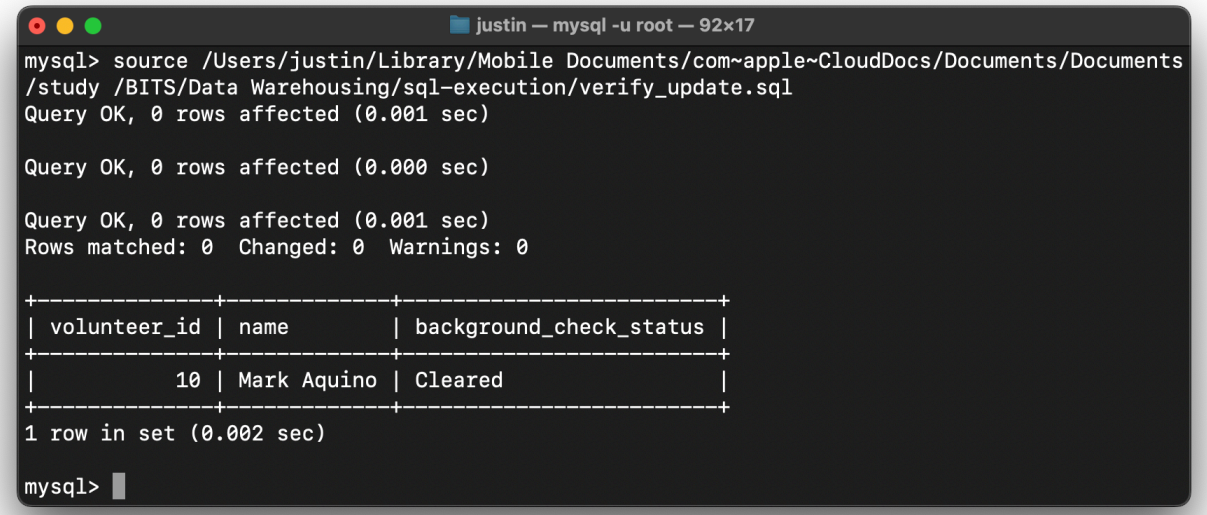


Output · before and after states for shelter 5

UPDATE Clearing a pending background check

This second UPDATE demonstrates a conditional update guarded by an additional predicate in the WHERE clause. The clause background_check_status = 'Pending' ensures the change only happens if the volunteer is in fact still pending, preventing accidental overwrites of records that have already been processed.

```
UPDATE Volunteer
SET background_check_status = 'Cleared'
WHERE volunteer_id = 10 AND background_check_status = 'Pending';

SELECT volunteer_id, name, background_check_status
FROM Volunteer
WHERE volunteer_id = 10;
```

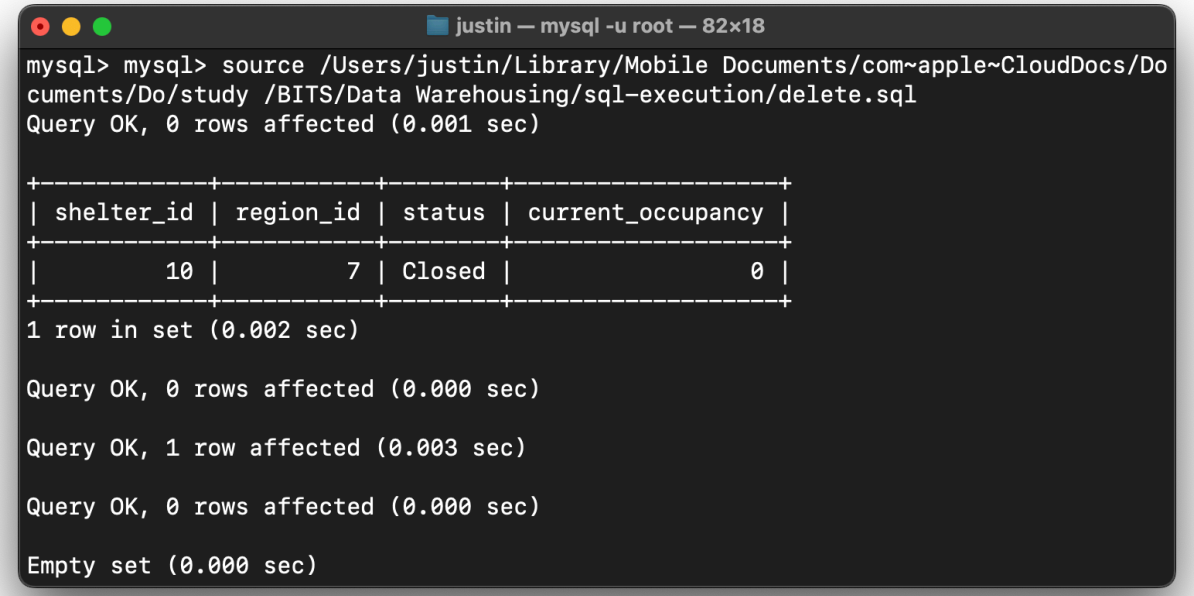


Output · volunteer 10 status updated to Cleared

DELETE Removing a decommissioned shelter

DELETE removes rows that satisfy the WHERE condition. Shelter 10 had a status of Closed and zero occupancy, making it safe to delete without orphaning any related records. A DELETE without a WHERE clause would empty the entire table, which is why the WHERE filter is treated as a non-negotiable safeguard in production environments.

```sql
-- Before
SELECT shelter_id, region_id, status, current_occupancy
FROM Shelter
WHERE status = 'Closed';

-- Delete
DELETE FROM Shelter
WHERE shelter_id = 10;

-- After (should return zero rows)
SELECT shelter_id, region_id, status, current_occupancy
FROM Shelter
WHERE shelter_id = 10;
```

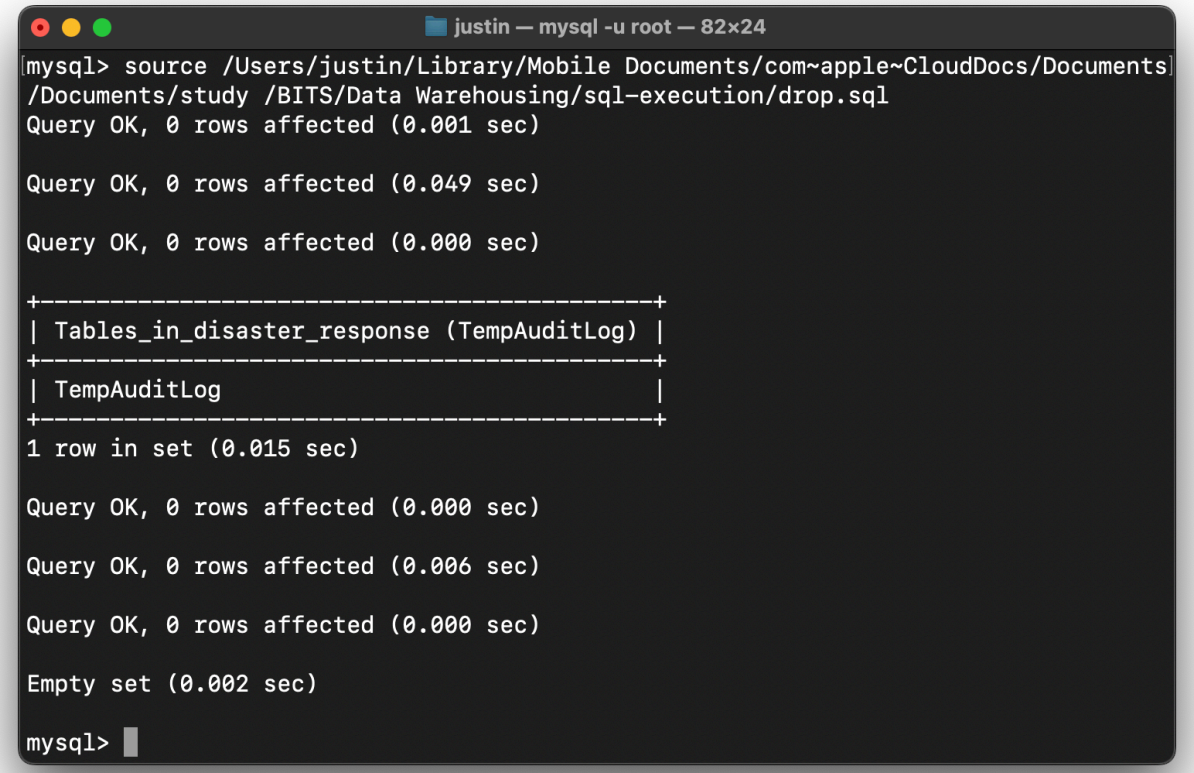


Output · shelter 10 removed; verification shows zero rows

DROP Removing a temporary audit table

DROP TABLE removes a table's structure and all its data permanently. To demonstrate this without losing project data, a throwaway audit table is created, verified, dropped, and verified again. DROP is destructive in a way DELETE is not: while DELETE empties a table, DROP destroys the table itself, including any indexes, constraints, and triggers attached to it.

```sql
CREATE TABLE TempAuditLog (
    log_id    INT PRIMARY KEY,
    action    VARCHAR(100),
    log_time  DATETIME
);

SHOW TABLES LIKE 'TempAuditLog';

DROP TABLE TempAuditLog;

SHOW TABLES LIKE 'TempAuditLog';   -- should return empty
```



Output · TempAuditLog created, then dropped

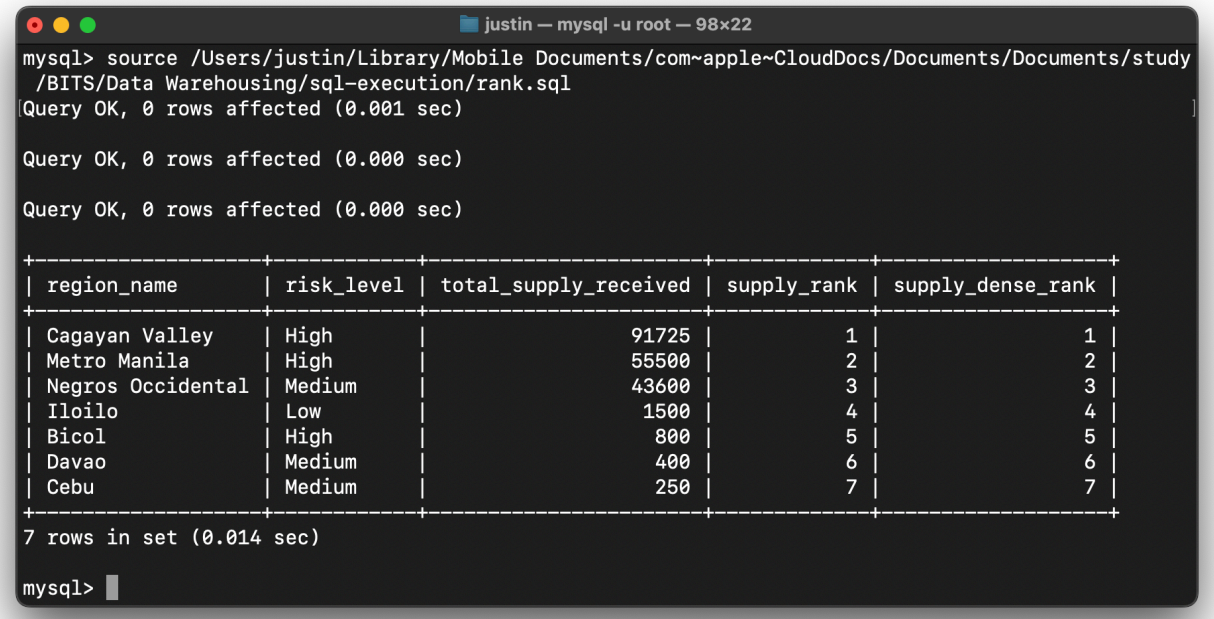
Common Table Expressions and Window Functions

This section is the project's *novel feature*. A Common Table Expression (CTE), introduced with the WITH keyword, defines a named temporary result set that can be referenced inside the main query. CTEs improve readability by separating the aggregation step from the analytical step. Window functions like RANK, DENSE_RANK, and NTILE then operate over a window of rows defined by an OVER clause, computing values that depend on the row's position within the window without collapsing the rows the way GROUP BY does.

WINDOW FN RANK and DENSE_RANK side by side

The CTE RegionSupplyTotals first aggregates total supplies received per region. The main query then applies two ranking functions side by side. RANK assigns the same rank to ties and skips the next rank or ranks (1, 2, 2, 4), while DENSE_RANK assigns the same rank to ties without skipping (1, 2, 2, 3). Showing both columns side by side makes the difference unambiguous and is the standard way to teach window function semantics. In this dataset there are no ties, so both columns return identical values; the difference would emerge if two regions received exactly the same total.

```sql
WITH RegionSupplyTotals AS (
    SELECT
        r.region_id,
        r.name AS region_name,
        r.risk_level,
        SUM(sa.quantity_allocated) AS total_supply_received
    FROM Region r
    JOIN SupplyAllocation sa ON r.region_id = sa.region_id
    GROUP BY r.region_id, r.name, r.risk_level
)
SELECT
    region_name,
    risk_level,
    total_supply_received,
    RANK()       OVER (ORDER BY total_supply_received DESC) AS supply_rank,
    DENSE_RANK() OVER (ORDER BY total_supply_received DESC) AS supply_dense_rank
FROM RegionSupplyTotals
ORDER BY supply_rank;
```

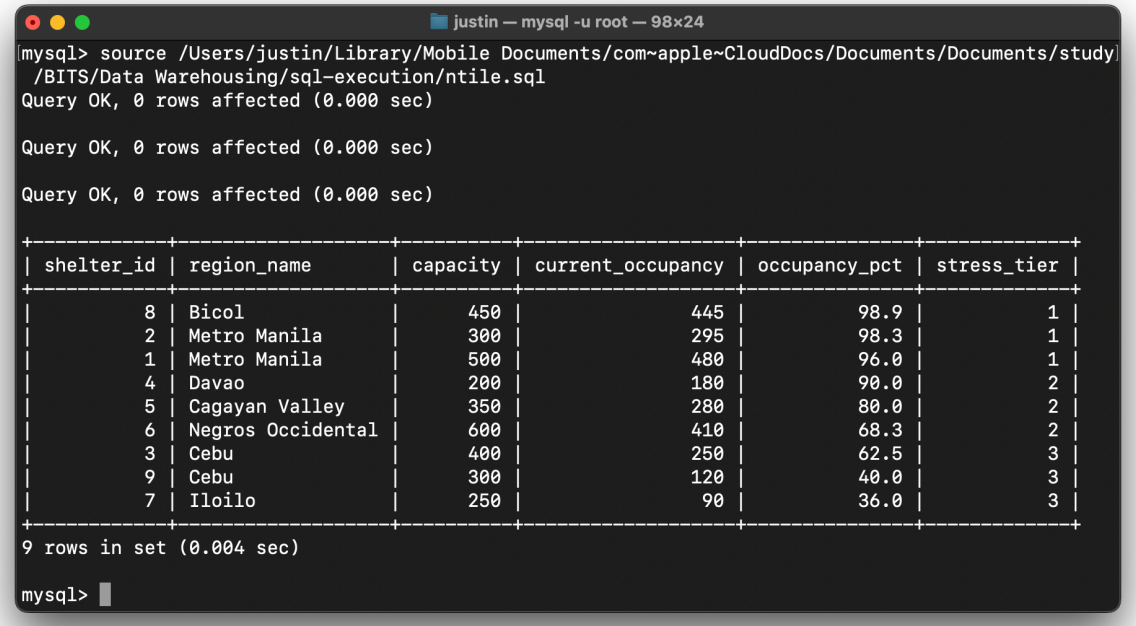


Output · regions ranked by total supplies received

WINDOW FN NTILE: bucketing shelters into stress tiers

NTILE divides the sorted rows into a specified number of approximately equal buckets. Here NTILE(3) splits the shelters into three stress tiers based on occupancy percentage, with tier 1 holding the most stressed shelters and tier 3 holding the least stressed. NTILE is fundamentally different from RANK in spirit: it is not ranking but bucketing, the kind of operation used for percentile analysis or for triaging where attention should go first when resources are limited.

```sql
WITH ShelterStress AS (
    SELECT
        s.shelter_id,
        r.name AS region_name,
        s.capacity,
        s.current_occupancy,
        ROUND((s.current_occupancy * 100.0 / s.capacity), 1) AS occupancy_pct
    FROM Shelter s
    JOIN Region r ON s.region_id = r.region_id
    WHERE s.capacity > 0
)
SELECT
    shelter_id,
    region_name,
    capacity,
    current_occupancy,
    occupancy_pct,
    NTILE(3) OVER (ORDER BY occupancy_pct DESC) AS stress_tier
FROM ShelterStress
ORDER BY stress_tier, occupancy_pct DESC;
```

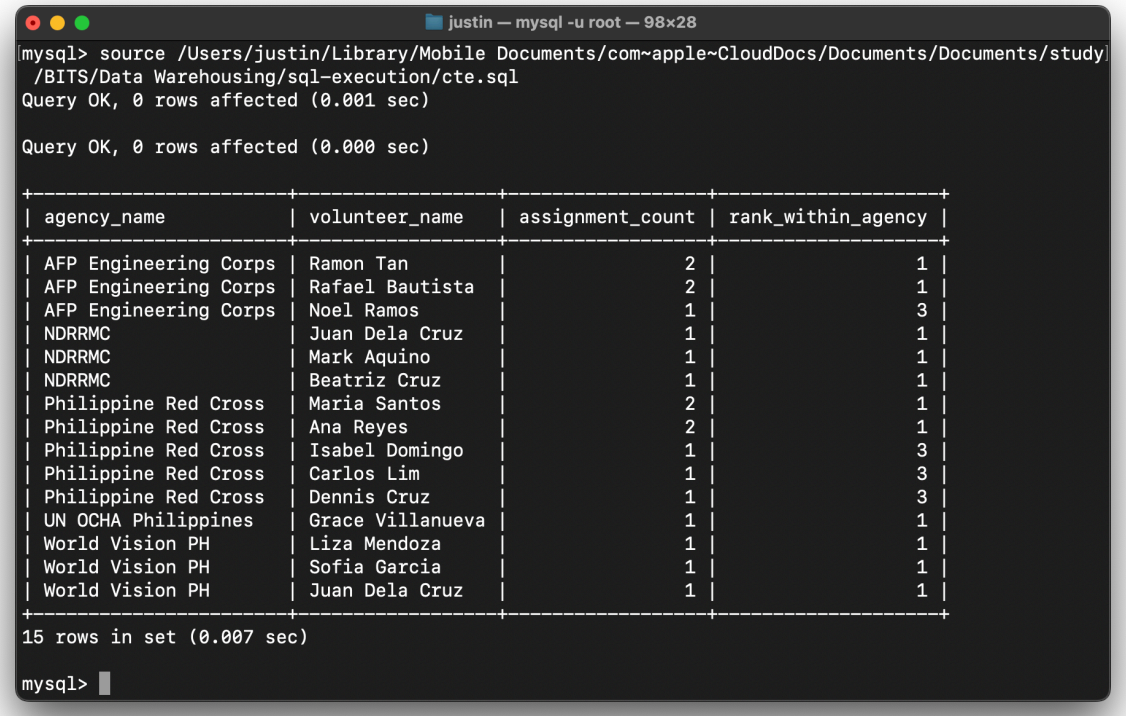


Output · nine shelters split cleanly into three tiers of three

WINDOW FN PARTITION BY: per-agency leaderboard

PARTITION BY divides the result set into independent groups before ranking is applied, so the rank counter resets at every new agency. This produces a per-agency leaderboard rather than one global leaderboard, which is the correct framing when the question is who is the most active volunteer for each agency. PARTITION BY is a heavily used pattern in real-world reporting (top product per category, top performer per region, top customer per market).

```sql
WITH VolunteerActivity AS (
    SELECT
        v.volunteer_id,
        v.name AS volunteer_name,
        a.name AS agency_name,
        COUNT(va.team_id) AS assignment_count
    FROM Volunteer v
    JOIN VolunteerAssignment va ON v.volunteer_id = va.volunteer_id
    JOIN ResponseTeam rt        ON va.team_id     = rt.team_id
    JOIN Agency a               ON rt.agency_id   = a.agency_id
    GROUP BY v.volunteer_id, v.name, a.name
)
SELECT
    agency_name,
    volunteer_name,
    assignment_count,
    RANK() OVER (PARTITION BY agency_name ORDER BY assignment_count DESC) AS rank_within_agency
FROM VolunteerActivity
ORDER BY agency_name, rank_within_agency;
```



Output · per-agency volunteer leaderboards

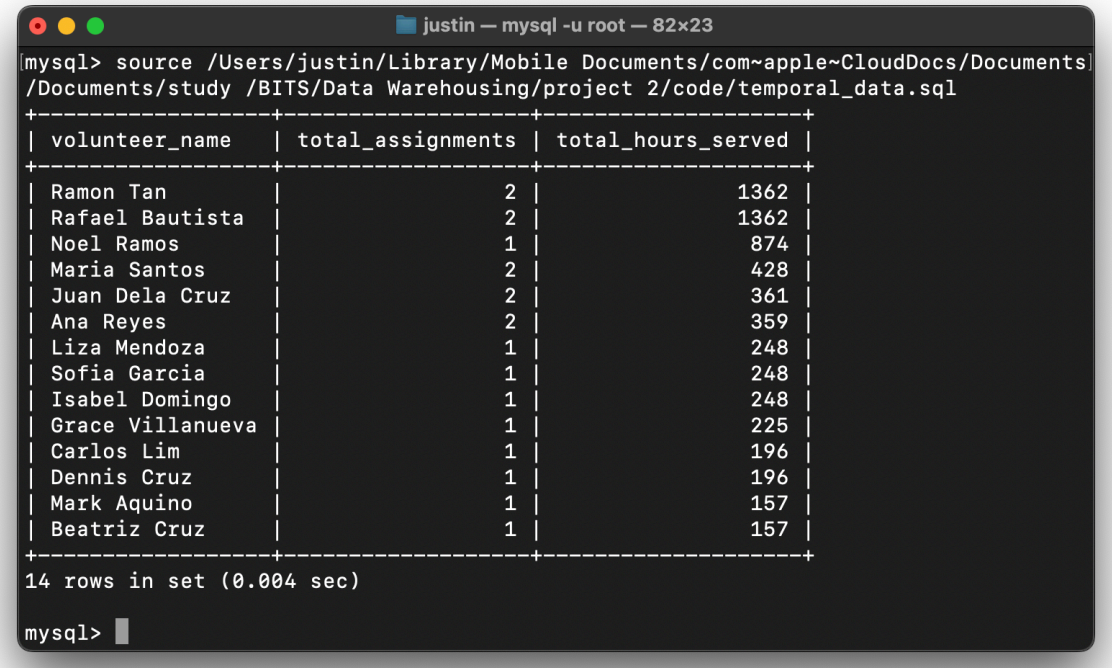
Working with Temporal Data

The schema invests heavily in temporal columns: assignment_start, assignment_end, deployment_start, deployment_end, and allocation_date. These exist so that questions about duration, overlap, and timing can be answered without leaving the database. The query below demonstrates the pattern.

Calculating volunteer hours served

This query uses *TIMESTAMPDIFF* to compute the elapsed hours between each volunteer's assignment start and end times, then aggregates across every assignment per volunteer. The result is a workload report ranked by total hours contributed, the kind of figure used for recognition, burnout monitoring, and post-deployment debriefs. *TIMESTAMPDIFF* is a MySQL function that returns the difference between two date-time values in a unit of the caller's choice (HOUR here, but DAY, MINUTE, and SECOND work the same way).

```sql
SELECT
    v.name AS volunteer_name,
    COUNT(va.team_id) AS total_assignments,
    SUM(TIMESTAMPDIFF(HOUR, va.assignment_start, va.assignment_end)) AS total_hours_served
FROM Volunteer v
JOIN VolunteerAssignment va ON v.volunteer_id = va.volunteer_id
GROUP BY v.volunteer_id, v.name
ORDER BY total_hours_served DESC;
```



Output · volunteers ranked by total hours served across all deployments

Conclusion

This project takes the Disaster Response Coordination System ER diagram from Project 1 and turns it into a fully working MySQL database with referential integrity enforced through foreign keys, business rules enforced through *CHECK* constraints, and a representative volume of sample data covering five disasters and seven affected regions. Every command category required by the rubric is demonstrated: *CREATE, INSERT, UPDATE, DELETE, DROP, and SELECT* for the core operations; *WHERE, LIKE, ORDER BY, GROUP BY, HAVING, and UNION* for filtering and aggregation; *INNER JOIN, LEFT JOIN, and RIGHT JOIN* for combining tables; a subquery using IN to filter against a derived list; and the novel feature, CTEs paired with the window functions *RANK, DENSE_RANK, NTILE, and PARTITION BY.*

Together these queries show that the database can answer the operational questions described in the original ERD presentation: which teams are deployed where and when, which regions are repeatedly affected and most resource-stressed, how supplies are distributed and where they are underused, and which volunteers are doing the most work for each agency. The structure scales: adding a new disaster requires only new rows in Disaster and DisasterRegion, with no schema change required, and every query in this document continues to work correctly.